

CSCE 645 GEOMETRIC MODELING · MODULE 04 · LECTURE NOTES

B-Splines and NURBS

Local control, automatic continuity, and exact circles.

MODULE OUTCOMES

- Explain local support and why it is the decisive property.
- Read a knot vector and predict the curve's behaviour from it.
- Explain how knot multiplicity controls continuity.
- Explain what weights add and why NURBS can represent conics exactly.
- Choose between Bézier, B-spline, and NURBS for a task.

1 What B-splines fix

Module 03 ended with three objections to high-degree Bézier curves: degree tied to control point count, global control, and oscillation. The proposed remedy was many low-degree segments with hand-maintained continuity constraints. B-splines are that construction with the constraints built into the basis.

Property	Bézier	B-spline
Degree	Fixed by control point count: $n+1$ points $\rightarrow$ degree n .	Chosen independently. Fifty control points, still cubic.
Support	Global — every control point affects every parameter.	Local — each affects $p+1$ spans only.
Continuity	Maintained by hand across joins.	Automatic C^{p-1} at every internal knot.
Endpoints	Interpolated.	Interpolated only if the knot vector is clamped.

$$\mathbf{C}(\mathbf{t}) = \sum_i \mathbf{N}_{i,p}(\mathbf{t}) \mathbf{P}_i$$

The form is identical to a Bézier curve — a weighted sum of control points. Only the basis functions differ. And critically, the B-spline basis functions are still non-negative and still sum to one at every parameter value, so the curve remains a convex combination of its control points. **Every property from Module 03 that followed from that fact carries over unchanged:** the convex hull property, affine invariance, variation diminishing. Only the support has become local.

2 Knot vectors

A B-spline is a piecewise polynomial, and the **knot vector** says where the pieces meet.

$$U = \{u_0, u_1, \dots, u_m\}, \quad \text{non-decreasing}, \quad m = n + p + 1$$

$$M = N + P + 1$$

Number of knots = number of control points + degree + 1.

This relation is the single most useful check when spline code misbehaves. An off-by-one in knot count produces curves that are mysteriously truncated, that do not reach their endpoints, or that evaluate to garbage near the ends.

Assert it on construction.

2.1 Uniform and clamped

- **Uniform:** knots evenly spaced, for example $\{0,1,2,3,4,5,6\}$. The curve does not reach its first or last control point — it starts and ends somewhere inside the control polygon, which is often surprising the first time.
- **Clamped:** the first and last knots repeated $p+1$ times, for example $\{0,0,0,0,1,2,3,3,3,3\}$ for a cubic. The curve now interpolates P_0 and P_n , recovering Bézier's endpoint behaviour. This is what you want almost always.

2.2 Multiplicity and continuity

$$\text{continuity at a knot of multiplicity } k = C^{p-k}$$

Multiplicity	Cubic ($p = 3$)	Appearance
1	C^2	Smooth, including curvature.
2	C^1	Tangent continuous; curvature breaks — visible in a reflection.
3	C^0	A corner . Position continuous only.
4	Discontinuous	The curve splits into separate pieces. Used at the ends to clamp.

ONE MECHANISM FOR BOTH SMOOTHNESS AND CORNERS

A designer wants a curve that is smooth everywhere except at one deliberate sharp point — a gear tooth, a letterform, a crease in a surface.

With Bézier segments this means splitting into separate objects and managing the join by hand. With a B-spline it means repeating one interior knot p times: the curve stays a single object with a single control polygon and has a corner exactly where you asked.

Clamping is the same mechanism at the ends. And a Bézier curve is a clamped B-spline with no interior knots — the general object contains the special one, which is the usual sign that the generalisation is the right one.

2.3 Evaluation: de Boor

```
vec3 de_boor(int k, float t, const std::vector<float>& U,
             std::vector<vec3> P, int p) {
    // k is the knot span: U[k] <= t < U[k+1].
    // Only control points k-p .. k participate -- local support.
    std::vector<vec3> d(P.begin() + k - p, P.begin() + k + 1);

    for (int r = 1; r <= p; ++r)
        for (int j = p; j >= r; --j) {
            float a = (t - U[j + k - p]) /
                      (U[j + 1 + k - r] - U[j + k - p]);
            d[j] = (1 - a) * d[j - 1] + a * d[j];
        }
    return d[p];
}
```

This is de Casteljau generalised: repeated linear interpolation, with the interpolation parameters derived from the knot vector rather than being t directly. Note that the loop touches only $p+1$ control points — local support is not an abstract property but a visible fact about which array elements are read.

3 NURBS

Non-Uniform Rational B-Splines. 'Non-uniform' refers to arbitrary knot spacing, already covered. 'Rational' is the new ingredient: each control point gains a positive weight.

$$\mathbf{C}(t) = [\sum w_i \mathbf{N}_{i,p}(t) \mathbf{P}_i] / [\sum w_i \mathbf{N}_{i,p}(t)]$$

With all weights equal to 1 the denominator is 1 and the curve reduces to an ordinary B-spline, so NURBS strictly generalise what came before. Raising a weight pulls the curve toward that control point, which is an intuitive handle for a designer.

The division is what matters. It makes the curve a *rational* function rather than a polynomial, and rational functions can do two things polynomials cannot.

3.1 Exact conics

NO POLYNOMIAL CURVE IS A CIRCLE

A circular arc is not the image of any polynomial map. Bézier and B-spline curves can only approximate one, and the error, while small, is real and accumulates through boolean operations and offsets.

A quadratic NURBS with weights (1, cos(θ/2), 1) on three control points is a circular arc, exactly. Likewise ellipses, parabolas, hyperbolas, and by extension cylinders, cones, spheres, and tori.

This is the reason CAD is built on NURBS. A manufactured cylindrical bore must be a cylinder; an approximation that is correct to six decimal places is still an approximation, and the errors compound when you intersect it with something.

3.2 Projective invariance

Module 03 noted that Bézier curves are affine invariant but not invariant under perspective projection, because perspective involves a division. Rational curves already involve a division, and that turns out to be exactly what is needed.

To transform a NURBS curve by a projective transformation, transform its control points in homogeneous form ($w_i \mathbf{P}_i$, w_i) and re-evaluate. The result is exactly the projected curve. This is the same mechanism as homogeneous coordinates in CSCE 641 Module 02 — the fourth coordinate and the final division — and it is the same payoff: a non-linear operation becomes linear by moving to a higher-dimensional space and dividing at the end.

4 Choosing

Representation	Use for	Because
Bézier	Font outlines, animation keyframes, short independent segments.	Simple, direct endpoint and tangent control, and ubiquitous in file formats.

Representation	Use for	Because
B-spline	Long editable curves, camera paths, profiles.	Local control makes editing practical; continuity is automatic.
NURBS	CAD, surfaces of revolution, anything with exact circular or conic features.	Exactness, projective invariance, and universal support in engineering software.
Subdivision	Characters, organic models, anything with arbitrary topology.	No patch layout or trimming required. Module 06.

WHY ENTERTAINMENT WENT ONE WAY AND ENGINEERING THE OTHER

NURBS are excellent for shapes that decompose naturally into rectangular patches — which engineered parts largely do.

A character's head does not. Covering an arbitrary closed surface with NURBS patches requires trimming curves and careful management of continuity across patch boundaries, both of which are difficult, fragile, and tedious to author.

Subdivision surfaces handle arbitrary topology with a single control mesh and no boundaries to manage, which is why Pixar adopted them in the 1990s and why every character pipeline since uses them.

This is a genuine split by application rather than one community being behind the other. CAD's requirement for exactness is real, and subdivision does not provide it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
A Primer on Bézier Curves — B-spline sections (free, interactive)	Knot vectors explained interactively, which is the only way they ever make sense the first time.
The NURBS Book — Piegl & Tiller (reference)	The standard reference. Check your library; the algorithms chapter is the one to consult when implementing de Boor and knot insertion.
Freya Holmér — The Continuity of Splines (free video)	An exceptionally clear visual treatment of continuity, knot multiplicity, and the spline families compared.
OpenNURBS / Rhino developer documentation	How NURBS are represented in a real CAD kernel, including trimming.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement B-spline evaluation with de Boor's algorithm. Verify that a clamped cubic with no interior knots reproduces a Bézier curve exactly.
- 2 Implement a knot vector validator that checks $m = n + p + 1$, non-decreasing order, and clamping. Deliberately break each condition and observe the failure mode.
- 3 Demonstrate local support: move one control point of a 20-point cubic B-spline and plot which parameter range changed. Confirm it spans exactly $p+1$ knot spans.
- 4 Vary the multiplicity of one interior knot from 1 to p and render each result. Capture the point at which a visible corner appears.
- 5 Implement NURBS evaluation. Construct an exact quarter circle with weights $(1, \cos 45^\circ, 1)$ and verify that sampled points lie on the circle to machine precision. Compare against a cubic Bézier approximation and report the maximum error.
- 6 Verify projective invariance: apply a perspective matrix to homogeneous control points and compare against projecting evaluated points. Then repeat with a non-rational B-spline and show it fails.
- 7 Implement knot insertion and verify it changes the control polygon without changing the curve.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three things B-splines fix relative to Bézier curves.
- 2 Why do the convex hull and affine invariance properties carry over unchanged?
- 3 State the relation between knot count, control point count, and degree, and say why it is worth asserting.
- 4 What is the continuity at a knot of multiplicity k , and how do you put a deliberate corner in a smooth B-spline?
- 5 What does 'rational' add, and what two things does it buy?
- 6 Why can no polynomial curve be exactly a circle, and why does that matter for CAD?
- 7 Why do character pipelines use subdivision rather than NURBS?